ENGINEERING PROJECT BLUEPRINT / CAMPUS PLACEMENT PORTFOLIO

# JobPulse

## Distributed Background Job Processing & Monitoring Platform

**Project purpose**
Build a full-stack platform where applications submit long-running jobs; independent workers execute them asynchronously; and users monitor progress, retries, failures, throughput, and results.

**Core stack** Node.js / Express.js / React / MongoDB / Redis / BullMQ / Docker
**Primary learning outcomes** Asynchronous architecture / Idempotency / Worker recovery / API security / Testing / Performance / DevOps

A detailed build reference containing product scope, system architecture, API contracts, database models, code samples, reliability design, security and testing requirements, a four-week implementation plan, and interview preparation.

**Important:** All API responses, dashboard values, metrics, and resume bullets in this document are templates or targets - not claims about an already-built project. Add only verified implementation details and benchmark results to your actual resume.

# Contents

1. Project overview and the problem it solves 3

2. Product scope: tasks and feature tiers 4

3. System architecture 5

4. End-to-end CSV workflow 6

5. Technology stack and learning priorities 7

6. Backend API design and response contracts 7

7. MongoDB models and job lifecycle 9

8. Job engine: producer, worker, and progress 11

9. Reliability: retries, idempotency, crash recovery 12

10. React dashboard: operational UI 13

11. Suggested repository structure 15

12. Security and access-control requirements 16

13. Testing and performance benchmarking 17

14. Four-week implementation schedule 18

15. Recruiter presentation and resume wording 19

16. Interview questions and concepts 20

17. Connections to RAG and future cloud work 21

18. Scope exclusions, first deliverable, and references 21

# 1. Project overview and the problem it solves

**Project statement.** Design and build a full-stack system that lets applications submit long-running tasks, executes them asynchronously through distributed workers, automatically retries recoverable failures, and provides real-time visibility into execution status and performance.

The project is intentionally backend-intensive. It demonstrates system design, asynchronous processing, fault tolerance, measurable performance, and operational visibility - not just a CRUD dashboard.

## Running example: a large CSV import

A user uploads a CSV containing 50,000 records. The application must validate rows, transform data, identify duplicates, persist accepted records, and produce a report. The process can take 30-60 seconds or longer depending on file size, workload, and hardware.

### Approach A: do all work inside the HTTP request

```
Client uploads CSV
    |
    v
Express request handler
    |
    +--> Validate --> Transform --> Database writes --> Report
    |
    v
Return response only after processing finishes
```

- The HTTP request may time out while the task runs.
- CPU-heavy synchronous work may block the Node.js event loop.
- Restarts can interrupt processing, and traffic bursts can exhaust resources.
- Users have no convenient mechanism for progress reporting, job history, or retrying failed tasks.

**Nuance:** A long HTTP request does not automatically block every other Node.js request. Asynchronous I/O can overlap, but synchronous CPU-intensive work or depleted shared resources can cause serious problems.

### Approach B: process the task asynchronously

1. The API validates the request, stores the uploaded file and metadata, and enqueues a job.
2. The API returns immediately with HTTP **202 Accepted** and a job identifier.
3. A separate worker picks up the job and executes it.
4. The worker records progress, produces results, and retries transient failures according to policy.
5. The client checks a status endpoint or receives live updates from the dashboard.

**What makes it a strong project**
Build a real CSV job and demonstrate the full lifecycle. Show that workers can fail, restart, and retry safely. Back your performance claims with reproducible tests rather than invented metrics.

# 2. Product scope: tasks and feature tiers

## Three supported job types

| Job type | What it does | Engineering value |
|---|---|---|
| Bulk CSV processing | Uploads a file, validates records, transforms data, inserts accepted rows, and generates a summary. | Batching, database writes, idempotency, progress tracking. |
| Report generation | Aggregates application records, creates a downloadable report, and stores a result reference. | I/O, long-running jobs, result lifecycle. |
| Failure simulation | Runs a controlled slow job, transiently fails, or triggers a permanent error. | Retry policies, recovery, monitoring, interview demos. |

**Suggested order:** Build CSV processing and failure simulation for the MVP. Add report generation after the processing core is reliable.

## Tier 1: Essential MVP

- Submit predefined jobs through a REST API.
- Store ownership and job metadata.
- Execute jobs in separate worker processes.
- Show waiting, active, completed, and failed states.
- Display progress, attempt counts, results, and errors.
- Retry temporary failures automatically.
- Build a React monitoring dashboard.
- Authenticate users and enforce ownership of jobs and files.

## Tier 2: Strong resume features

- Run multiple worker processes with configurable concurrency.
- Use exponential backoff and sensible retry classifications.
- Implement idempotency and duplicate-job protection.
- Demonstrate worker crash recovery.
- Provide job history and execution timelines.
- Run the services using Docker Compose.
- Add automated API/integration tests.
- Measure throughput and latency using load tests.

## Tier 3: Advanced differentiators

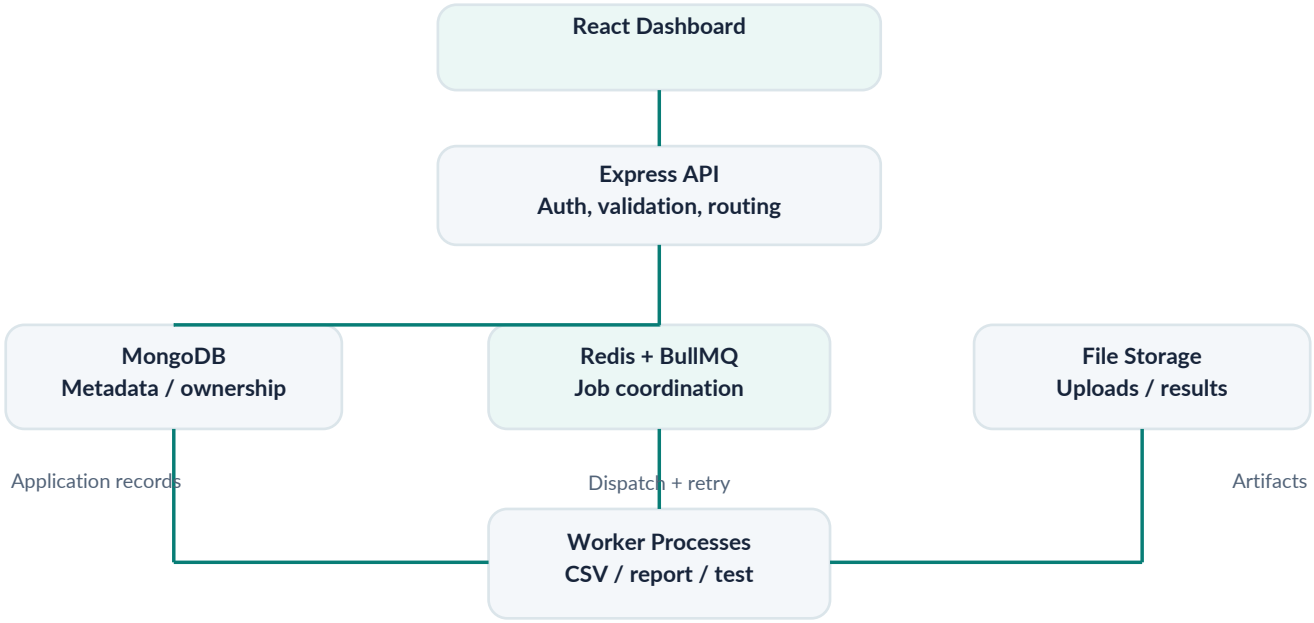
- Job priorities, delayed and recurring jobs.
- Cooperative cancellation.
- Queue pausing and restricted admin controls.
- Server-Sent Events or WebSockets for live updates.
- Prometheus / Grafana or comparable observability.

- Cloud deployment and CI/CD.
- Permanent-failure review workflow or dead-letter handling.
- Per-user rate limits and resource quotas.

All advanced features are optional until the MVP is stable. Scope control is an engineering decision in its own right.

# 3. System architecture

Separate responsibilities explicitly: the API accepts and authorizes requests; the queue coordinates execution; workers perform work; MongoDB stores application records; and file storage holds large artifacts.




*Conceptual architecture. Arrows indicate the main responsibilities, not every event or bidirectional communication path. A single worker is sufficient initially; scale to multiple processes later.*

### Component responsibilities

| Component | Responsibilities |
|---|---|
| React dashboard | Submit jobs; show queue summary, filtering, progress, error details, and results. |
| Express API | Authentication, request validation, authorization, upload and submission endpoints, job queries. |
| MongoDB | Users, file references, job ownership, application metadata, and durable historical summaries. |
| Redis | BullMQ state and coordination between producers and workers. |
| BullMQ | Queue, worker registration, retries, scheduling, progress, and job state coordination. |
| Workers | Independent processes that execute task handlers and report their outcomes. |
| Task handlers | CSV import, report generation, and controlled failure simulation. |
| File storage | Uploaded CSV files and generated reports. Start with durable local volumes, then consider object storage. |
| Monitoring | Job events, queue counts, waiting time, processing durations, failures, and logs. |

### Why BullMQ instead of writing a custom broker?

BullMQ provides queueing and worker coordination, including retries, concurrency, delayed execution, and recovery of stalled jobs. You still design and implement the application-specific parts: processor semantics, API contracts, authorization, consistency, operational controls, tests, and measurements.

Core concepts include **Queue**, **Worker**, **QueueEvents**, and optionally **FlowProducer** for dependent jobs. Using an established queue engine is a realistic architecture choice, not a shortcut to avoid engineering.

## 4. End-to-end CSV workflow

1. **Upload:** The user uploads a CSV; the API checks authentication, ownership, file limits, type, and safe storage.
2. **Persist metadata:** Store a file reference and a submission record.
3. **Enqueue:** Add a BullMQ job containing the job type and references to data. Avoid putting large file contents into Redis job payloads.
4. **Acknowledge:** Return HTTP 202 with a job ID and status URL.
5. **Execute:** A worker streams or reads the file in manageable batches, validates rows, and performs safe database writes.
6. **Report progress:** Periodically update percent complete or processed/total counts.
7. **Finish or fail:** Produce a summary and result reference, or retry only recoverable failures.
8. **Show results:** Display totals, rejected rows, duration, download links, and errors in the dashboard.

**Core architectural decision**
The worker, not the HTTP request handler, owns the long-running processing. The API owns authorization, validation, and submission. The result is a short-lived request and a separately managed task.

# 5. Technology stack and learning priorities

| Technology | Role | Priority |
|---|---|---|
| JavaScript / Node.js | API and worker runtime; event-loop behaviour. | Deepen existing knowledge |
| Express.js | HTTP routes, middleware and validation. | Already familiar |
| React + Vite | Dashboard and job detail views. | Already familiar |
| MongoDB / Mongoose | Application metadata and ownership. | Already familiar |
| Redis | Backing store for job coordination. | New: essential |
| BullMQ | Queue, worker, retry, and scheduling APIs. | New: essential |
| Docker Compose | Local multi-service development. | Intermediate |
| Vitest / Supertest | Unit tests and API/integration tests. | Intermediate |
| k6 | HTTP load testing; complement with job workload script. | Intermediate |
| GitHub Actions | Test/build automation. | Intermediate |
| SSE or WebSockets | Push live dashboard updates. | Advanced |
| Prometheus / Grafana | Metrics and dashboards. | Advanced |
| AWS | Optional public deployment. | Advanced |

Start with one Express API, one worker process, Redis, MongoDB, and React. Do not add Python, Kubernetes, LangChain, arbitrary code execution, or microservices unless requirements genuinely demand them.

# 6. Backend API design and response contracts

## Proposed REST endpoints

Use the prefix **/api/v1** throughout. Protect all job-specific routes with ownership checks; do not rely on authentication alone.

| Method | Route | Purpose |
|---|---|---|
| POST | /auth/register | Register user |

| Method | Route | Purpose |
|---|---|---|
| POST | /auth/login | Authenticate user |
| POST | /files | Upload CSV and return a file ID |
| POST | /jobs | Submit background job |
| GET | /jobs | List current user's jobs |
| GET | /jobs/:id | Fetch job status and metadata |
| GET | /jobs/:id/events | Get execution history (later) |
| GET | /jobs/:id/result | Get or download job output |
| POST | /jobs/:id/retry | Manually retry eligible failures |
| POST | /jobs/:id/cancel | Request cooperative cancellation (later) |
| GET | /dashboard/summary | Queue counts and user-level metrics |
| GET | /health | Readiness/liveness checks |

## Request example: submit a CSV import

First upload the file to obtain its ID; then submit the job using an authenticated request.

```
POST /api/v1/jobs
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "csv-import",
  "fileId": "file_123",
  "options": {"skipInvalidRows": true}
}
```

### Response example: HTTP 202 Accepted

```
{
  "jobId": "job_123",
  "status": "queued",
  "message": "Job accepted for processing",
  "statusUrl": "/api/v1/jobs/job_123"
}
```

### Fetch job status

```
GET /api/v1/jobs/job_123

{
  "jobId": "job_123",
  "type": "csv-import",
  "status": "active",
  "progress": 65,
  "attemptsMade": 1,
  "createdAt": "2026-11-02T10:00:00Z",
  "startedAt": "2026-11-02T10:00:03Z",
  "result": null
}
```

#### Completed-job result example

```
{
  "jobId": "job_123",
  "status": "completed",
  "progress": 100,
  "result": {
    "totalRows": 5000,
    "processedRows": 4975,
    "rejectedRows": 25
  }
}
```

IDs, timestamps, row totals, and status values above are illustrative. Design consistent validation and error responses (e.g., 400 for invalid payload, 401 for unauthenticated, 403 or privacy-preserving 404 for unauthorized, 404 for nonexistent, 503 when submission infrastructure is unavailable).

# 7. MongoDB models and job lifecycle

### User collection

```
User
  _id
  name
  email
  passwordHash
  createdAt
```

## File collection

```
File
  _id
  ownerId
  originalName
  storageKey
  size
  mimeType
  createdAt
  metadata
```

The storage key references a file stored outside the job payload. Do not embed large CSVs in Redis. Validate user authorization whenever a file is read or referenced by a job.

## JobRecord collection

```
JobRecord
  _id
  ownerId
  bullmqJobId
  type
  inputFileId
  idempotencyKey
  enqueueStatus
  createdAt
  completedAt
  resultReference
  resultSummary
```

## State transitions

```
CREATED --> WAITING --> ACTIVE --> COMPLETED
                        |
                        +--> TEMPORARY FAILURE --> DELAYED --> WAITING
                        |
                        +--> PERMANENT / EXHAUSTED FAILURE --> FAILED
                                                               |
                                                               +--> MANUAL RETRY
```

This is a simplified model. BullMQ includes additional states, and stalled jobs can be recovered or fail depending on the detection and retry configuration.

## Avoid competing sources of truth

Treat BullMQ/Redis as the authority for current execution state. Store ownership, metadata, and durable historical summaries in MongoDB. The API can combine these for responses. Persist final results before queue record retention policies remove old jobs.

A cross-store failure can occur when MongoDB creates a record but Redis enqueueing fails (or vice versa). A production design needs an enqueue status, retry/reconciliation mechanism, and idempotent submission logic. This is an excellent system-design topic to document explicitly.

# 8. Job engine: producer, worker, and progress

## Producer: add jobs to BullMQ

```
import { Queue } from "bullmq";

const jobQueue = new Queue("jobpulse", {
  connection: { host: "localhost", port: 6379 }
});

await jobQueue.add("csv-import", {
  fileId: "file_123",
  ownerId: "user_123"
}, {
  jobId: "job_123",
  attempts: 3,
  backoff: { type: "exponential", delay: 2000 }
});
```

This is an illustrative skeleton. **attempts: 3** means up to three total attempts, not three retries. Use the configured job ID carefully: a deduplication strategy should not assume that it protects an external database operation from duplicate side effects.

## Worker: execute job handlers in a separate process

```
import { Worker } from "bullmq";

const worker = new Worker("jobpulse", async (job) => {
  switch (job.name) {
    case "csv-import":
      return processCSV(job);
    case "report-generation":
      return generateReport(job);
    case "failure-simulation":
      return simulateFailure(job);
    default:
      throw new Error("Unsupported job type");
  }
}, {
  connection: { host: "localhost", port: 6379 },
  concurrency: 3
});
```

The processor functions must be implemented separately. Begin with concurrency 1. Test concurrency 3 or 5 after the basic path works, then compare multiple separate worker processes. For CPU-heavy JavaScript, greater in-process concurrency does not imply parallel CPU execution; investigate sandboxed processors or dedicated processes.

## Worker progress

```
// Example inside a processor after completing a batch:
await job.updateProgress(25);
```

Report progress periodically, not once per row. For a CSV job the UI might show "2,500 / 5,000 rows, 50%". Progress numbers should reflect real work and should tolerate retries resetting or revisiting work.

# 9. Reliability: retries, idempotency, crash recovery

## A. Retry policies and exponential backoff

Recoverable failures include transient network errors, temporary database outages, and some HTTP 503/429 responses. Invalid payloads, unsupported file types, and permission errors should generally fail without repeated processing.

| Failure | Typical action |
|---|---|
| Temporary network problem | Retry with bounded attempts and backoff. |
| HTTP 503 | Retry with backoff and/or server retry guidance. |
| Rate limiting (429) | Honor rate-limit guidance; avoid synchronized retry storms. |
| Unsupported file format | Do not retry automatically. |
| Invalid user input | Do not retry automatically. |
| Permission denied | Do not retry automatically. |
| Temporary database unavailability | Consider retry; ensure re-execution is safe. |

A backoff base of 2 seconds can produce roughly 2s, 4s, 8s between successive retries if enough total attempts are permitted. Tune attempts and delays for the task rather than copying one policy everywhere.

## B. Idempotency: avoid duplicate side effects

Suppose a worker successfully inserts 3,000 rows and crashes before acknowledging completion. A retry can process those rows again. "At least once" execution means processors must tolerate repeated attempts.

- Give each import a stable operation or import ID.
- Derive a stable row identity, e.g. (importId, rowNumber) when appropriate.
- Enforce a compound unique index in MongoDB.
- Use safe upserts or idempotent database writes.
- Optionally checkpoint meaningful progress; ensure checkpoints and side effects stay consistent.

**Correct guarantee to explain**
The goal is safe repeated execution with no unintended duplicated effects. It is not accurate to promise that a distributed worker job will only ever execute once.

## C. Worker crash recovery experiment

1. Start at least two worker processes.

2. Submit a controlled long-running job.
3. Terminate the process currently executing the job.
4. Observe stalled-job handling and later retry/recovery while another worker is available.
5. Inspect queue events and final result state.
6. Verify that retries have not created duplicate records or outputs.

Recovery may wait for a lock to expire and stalled-job detection to run. Redis persistence settings, limits, and application idempotency affect durability. Do not claim instantaneous or unconditional recovery.

### D. Multiple workers and concurrency

Compare one versus multiple processes with a fixed workload. Observe competing consumers, queue depth, database contention, CPU/memory, and throughput. More concurrency can help I/O-bound work but may worsen resource contention or add no benefit for CPU-bound tasks.

### E. Scheduling, priorities, and cancellation

After the MVP, introduce higher priority jobs, delayed scheduling, recurring cleanup/reporting, and restricted pause/resume controls. Cancellation should be cooperative: a worker checks for a cancellation signal at safe interruption points, particularly between CSV batches.

## 10. React dashboard: operational UI

The frontend should feel like an operational console, not a generic list of tasks. Every UI control should reflect real backend state and permissions.

### Recommended pages

| Page | What it shows |
|---|---|
| Overview | Queue counts, success rate, recent activity, throughput and failures. |
| Submit Job | Approved job types, upload control, safe configuration fields. |
| Job History | Search, filter, pagination, ownership-aware status list. |
| Job Details | Progress, attempt count, event timeline, logs, result and error details. |
| Queue Operations | Restricted pause/resume, manual retry, administrative actions. |
| Performance | Queue wait time, execution duration, throughput, latency trends. |

## Illustrative dashboard information hierarchy

```
JOBPULSE                            [Submit New Job]
-----------------------------------------------------
Total Jobs: --   Running: --   Completed: --   Failed: --

Job History
-----------------------------------------------------
Type                Status       Progress    Attempts
CSV Import          Active          65%          1
Report Generation   Completed      100%          1
Failure Simulation  Failed           --          3

[View details] [Filter by status] [Search] [Next page]
```

These are fictional UI values for design illustration only. Start with polling every few seconds. Add Server-Sent Events or WebSockets only when the core job system is stable.

# 11. Suggested repository structure

```
jobpulse/
|-- client/
|   `-- src/
|       |-- components/
|       |   |-- JobTable.jsx
|       |   |-- JobCard.jsx
|       |   |-- StatusBadge.jsx
|       |   `-- ProgressBar.jsx
|       |-- pages/
|       |   |-- Dashboard.jsx
|       |   |-- SubmitJob.jsx
|       |   |-- JobDetails.jsx
|       |   `-- Login.jsx
|       |-- services/
|       `-- hooks/
|-- server/
|   `-- src/
|       |-- controllers/
|       |-- routes/
|       |-- models/
|       |   |-- User.js
|       |   |-- File.js
|       |   `-- JobRecord.js
|       |-- services/
|       |   |-- jobService.js
|       |   `-- fileService.js
|       |-- middlewares/
|       |-- validators/
|       `-- app.js
|-- worker/
|   `-- src/
|       |-- processors/
|       |   |-- csvProcessor.js
|       |   |-- reportProcessor.js
|       |   `-- failureSimulator.js
|       `-- worker.js
|-- shared/
|   |-- queues/
|   |   `-- jobQueue.js
|   `-- jobTypes.js
|-- tests/
|   |-- api/
|   |-- integration/
|   `-- load/
|-- docs/
|   |-- architecture.md
|   |-- api-spec.md
|   `-- benchmarks.md
|-- docker-compose.yml
|-- .env.example
`-- README.md
```

This is a **target organization**, not a requirement to create empty folders on Day 1. Start small and create modules when there is actual responsibility to isolate. Keep configuration and task contracts shared where it avoids duplicated logic.

# 12. Security and access-control requirements

| Risk | Recommended defence |
|---|---|
| Unauthenticated requests | Protect job and file routes with secure session or JWT authentication. |
| Cross-user access | Enforce ownership on every job, file, result, and event request. |
| Unsupported task payloads | Allowlist job types and validate options against schemas. |
| Malicious or oversized uploads | Size limits, file-type checks, safe paths, streaming, storage quotas. |
| Resource exhaustion | Rate limits, per-user submission quotas, bounded concurrency. |
| Sensitive data exposure | Avoid secrets and PII in job payloads, logs, and public error responses. |
| Public Redis / MongoDB | Private network binding, credentials and environment configuration. |
| Unauthorized admin controls | Role-based permissions for queue pause, retry-all and monitoring. |

**Deliberate exclusion:** Do not allow untrusted users to submit arbitrary JavaScript or shell commands. Executing arbitrary code safely is a separate, much harder sandboxing/security problem.

Reference: OWASP API Security Top 10, including broken object-level authorization and unrestricted resource consumption.

# 13. Testing and performance benchmarking

## Functional and failure tests

| Test | Expected behaviour |
|---|---|
| Valid CSV job | Accepted, queued, processed, result available. |
| Invalid job type | Appropriate validation response. |
| Unauthenticated submission | Rejected with 401. |
| Access another user's job | Rejected without disclosing private data. |
| Normal completion | Correct records, final state, and result summary. |
| Transient failure | Retries within configured policy. |
| Permanent validation failure | Fails without unnecessary retries. |
| Worker crash | Eventually recovered or correctly failed under policy. |
| Repeated execution | No unintended duplicate records. |
| All workers stopped | Pending work waits until processing resumes. |
| Redis unavailable | API does not falsely claim successful queue submission. |

Use unit tests for validators and processors, API tests for request/response contracts, integration tests with Redis and MongoDB, and end-to-end tests for genuine CSV jobs. Keep some failure experiments reproducible with documented steps.

## Performance experiments

Use a fixed dataset and repeat tests with one, two, and four worker processes. Measure API submission separately from job processing: fast submission responses do not necessarily mean fast completed jobs.

| Metric | What it means |
|---|---|
| API p95 latency | 95th-percentile submission/query response time. |
| Queue wait time | Time between enqueueing and a worker starting. |
| Processing duration | Execution time while the worker runs. |
| End-to-end completion | Submission-to-result elapsed time. |
| Throughput | Jobs or records successfully processed per unit time. |
| Retry count | Frequency of recoverable job failures. |
| Ultimate failure rate | Fraction of jobs that exhaust their attempts. |

| Metric | What it means |
|---|---|
| Queue depth | Backlog and ability to keep up with arrivals. |
| CPU / memory | Potential worker, database, or Redis bottlenecks. |

### Benchmark report template

| Worker setup | Test workload | Throughput | p95 wait | Error rate |
|---|---|---|---|---|
| 1 worker | Same fixed CSV set | Record actual | Record actual | Record actual |
| 2 workers | Same fixed CSV set | Record actual | Record actual | Record actual |
| 4 workers | Same fixed CSV set | Record actual | Record actual | Record actual |

Record machine specifications, Node/Redis configuration, dataset size, database condition, number of iterations, and measurement method. Do not invent throughput or latency numbers. More workers do not guarantee linear speedups.

## 14. Four-week implementation schedule

This is an ambitious MVP-plus-polish schedule for approximately 15-20 focused hours per week. If campus interviews or exams interrupt the plan, prioritize completing the MVP rather than adding partially finished features.

### Week 1 - Asynchronous processing foundation

- Write the project requirements, job contracts, and lifecycle diagram.
- Start Redis, MongoDB, Node.js, and BullMQ.
- Create the submission API and status endpoint.
- Run a standalone worker and implement failure simulation.
- Build a small React status page.

**Demo:** Submit 10 test jobs and watch workers process them; inspect final outcomes.

### Week 2 - Real CSV tasks and recovery

- Implement CSV upload, validation, batch processing, and progress reporting.
- Configure bounded retries and exponential backoff.
- Run multiple workers and observe results.
- Implement authorization and user ownership.
- Create initial integration tests.

**Demo:** Process an actual CSV and show recoverable errors being retried.

### Week 3 - Reliability and reproducibility

- Implement idempotency using stable identifiers and uniqueness rules.

- Run crash/restart experiments.
- Add report generation if core functionality is stable.
- Build job history and result download support.
- Set up Docker Compose and increase test coverage.

**Demo:** Crash a worker during an import and verify no unintended duplicate records.

### Week 4 - Benchmark, deploy, and present

- Load-test submission and processing with controlled workloads.
- Compare throughput and wait time across worker configurations.
- Add basic monitoring and structured logs.
- Run tests automatically in GitHub Actions.
- Deploy a small demonstrable version if cost and time permit.
- Write README, architecture notes, benchmarks, and a short walkthrough.

**Demo:** Show a working product, a documented failure test, and measured performance results.

### Minimum resume-worthy milestone

**Do not wait for optional cloud features**
One working useful job, Redis-backed worker execution, job status UI, retry handling, ownership checks, reproducible setup, and meaningful automated tests are enough to make a credible first portfolio version.

## 15. Recruiter presentation and resume wording

### README structure

1. What problem JobPulse solves and why asynchronous processing is useful.
2. System architecture diagram and component responsibilities.
3. Prerequisites, installation, environment variables, and quick-start commands.
4. Supported job types and sample API requests.
5. Job lifecycle, retries, idempotency, and crash-recovery details.
6. Screenshots or demonstration video and sample CSV input.
7. Testing instructions, benchmark methodology, and measured results.
8. Known limitations, future work, and security assumptions.

### Example resume entry - only after implementation

**JobPulse | Distributed Job Processing & Monitoring Platform**
**Stack:** Node.js, Express.js, Redis, BullMQ, MongoDB, React, Docker

1. Engineered an asynchronous background-processing platform using Node.js, BullMQ, and Redis to execute CSV-processing and report-generation jobs independently of HTTP requests.

2. Implemented configurable concurrency, retries with exponential backoff, job-progress tracking, and failure-recovery tests.

3. Developed a React monitoring dashboard for inspecting job states, execution history, errors, and processing results.

4. Containerized application services and conducted controlled load tests to evaluate job throughput, queue latency, and worker scalability.

The resume text is a **target template**, not current experience. Remove bullets for anything not implemented. Add exact metrics only when you can reproduce the experiment and defend its conditions in an interview.

## 16. Interview questions and concepts

| Question | Core concepts |
|---|---|
| Why not process everything inside Express? | Event loop, HTTP timeouts, blocking work, async processing. |
| Why choose Redis? | In-memory data structures, coordination, persistence, trade-offs. |
| Why use BullMQ rather than your own queue? | Engineering abstractions, reliability, scope control. |
| What if a worker crashes? | Locks, stalled jobs, retries, recovery limits. |
| Could a task run twice? | At-least-once delivery, idempotent processors. |
| How do you scale throughput? | Concurrency, worker count, bottlenecks, resource contention. |
| Why not Promise.all() for every job? | Backpressure, concurrency bounds, durability and resource use. |
| Why exponential backoff? | Retry storms, transient errors, rate limiting. |
| How are private jobs protected? | Authentication, authorization, ownership enforcement. |
| How do you measure scale? | Queue depth, throughput, p95 latency, error rates. |
| How are workers deployed? | Processes, Docker, config, restart and health policies. |
| How do you keep results consistent? | Transactions, idempotency, cross-store reconciliation. |

**Preparation method:** For each topic, learn the concept, implement it, deliberately trigger its failure mode, and practise explaining the trade-off in your own words.

# 17. Connections to RAG and future cloud work

JobPulse is intentionally not an AI project initially. But its queue-and-worker architecture transfers directly to document ingestion in a RAG system.

```
PDF upload
    |
    v
API creates ingestion job
    |
    v
Redis / BullMQ queue
    |
    v
Document worker
    |
    +--> Extract text
    +--> Split into chunks
    +--> Generate embeddings
    +--> Store vector representations
    |
    v
Document ready for RAG retrieval and question answering
```

A future application can process PDFs in workers and perform interactive retrieval in the API. You can later learn distributed tracing and telemetry across the API and workers using OpenTelemetry, and deploy to cloud infrastructure after the local system is well tested.

# 18. Scope exclusions, first deliverable, and references

## Do not build these in the first version

- A custom Redis replacement or queue broker.
- A drag-and-drop workflow editor.
- Kubernetes-based microservices.
- Arbitrary user-provided shell/JavaScript execution.
- Multi-region deployments.
- An AI agent that manages the queue.
- Complex payment, subscription, or billing infrastructure.

These greatly expand the scope without necessarily improving campus interview outcomes. Be clear that BullMQ supplies the queue implementation and JobPulse supplies the surrounding system.

## First engineering deliverable: docs/architecture.md

1. What are the API responsibilities versus worker responsibilities?
2. What belongs in MongoDB versus Redis?

3. What does the submission endpoint return immediately?
4. What happens if a CSV import fails halfway through?
5. How will retries avoid duplicate database records?

Then build the smallest proof of concept: **one Express route + one BullMQ queue + one independent worker + one simple task**. Once that works, add a real CSV processor and progressively build the platform.

## Official documentation and learning references

- BullMQ official documentation https://docs.bullmq.io/
- Redis documentation https://redis.io/docs/latest/
- Docker Compose https://docs.docker.com/compose/
- Grafana k6 performance testing https://grafana.com/docs/k6/latest/
- OpenTelemetry JavaScript https://opentelemetry.io/docs/languages/js/
- OWASP API Security Top 10 https://owasp.org/API-Security/editions/2023/en/0x11-t10/
- Node.js event loop and worker pool https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop
- GitHub Actions documentation https://docs.github.com/en/actions

**Final engineering goal**
JobPulse should demonstrate that you can take a backend problem, design the architecture, independently implement it, handle failures, verify correctness, measure performance, and discuss trade-offs. Those skills transfer to stronger SDE interviews, cloud engineering, and more advanced RAG systems.